

CMSE 801 - Day 18



Building an agent-based model for modeling
forest fires

November 4, 2024

General Course Announcements

- Homework 4 is available and due November 15 by 11:59pm.

Questions/comments from Day 18 PCA

- Most common struggles were with the last part, trying to set up the forest/"board" -- **we'll look at that together and talk it through.**
 - Specifically, trying to understand the right way to set up the nested loops
- Some questions about the function that plots the board and what `np.meshgrid` is doing. **We'll review this as well.**

Setting up the "forest"

```
In [1]: # First get some imports out of the way  
import random  
import numpy as np  
import matplotlib.pyplot as plt
```

Setting up the "forest" (cont'd)

```
In [2]: def create_forest(N=50,f_trees_start=0.5):  
    ...  
    Creates the initial game board.  
    Inputs:  
        board_size: length of one edge of the board  
        f_trees_start: probability that a given cell is a tree  
                       (effectively the tree density)  
  
    Outputs a 2D numpy array with values set to either 0, 1, or 2  
        (empty, tree, or fire)  
    ...  
  
    # all cells initialized to 'empty' (0) by default; board is defined  
    game_board = np.zeros((N,N))  
  
    # loop over all rows and columns on the board and roll the dice;  
    # if the random number is less than f_trees_start, make it a tree.  
    for i in range(game_board.shape[0]): # .shape[0] gives us number of  
        for j in range(game_board.shape[1]): # .shape[1] gives us the n  
            if random.random() <= f_trees_start:  
                game_board[i,j] = 1  
  
    # set the whole left edge of the board on fire.  
    # This means all rows for the first column, should be set to 2  
    game_board[:,0] = 2  
  
    return game_board
```

Visualizing the "forest"

```
In [3]: # function plotgrid() takes in a 2D array and uses matplotlib to make a
# this function returns no values!

def plotgrid(myarray):

    # first create two vectors based on the x and y sizes of the grid
    x_range = np.linspace(0, myarray.shape[1]-1, myarray.shape[1]) # x r
    y_range = np.linspace(0, myarray.shape[0]-1, myarray.shape[0]) # y r

    # use the numpy meshgrid function to create two matrices
    # of the same size as myarray with x and y indexes
    x_indexes, y_indexes = np.meshgrid(x_range, y_range)

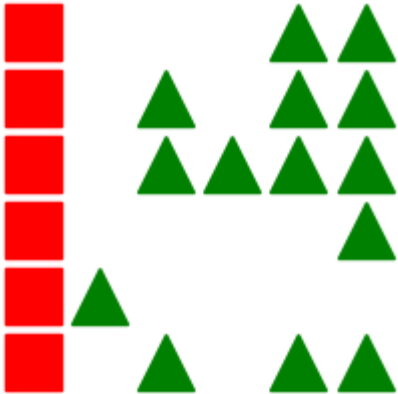
    # make a list of all the x and y indexes that are either squares or
    # the notation below is relatively new to us; it means that when mya
    # only record those values.
    tr_x = x_indexes[myarray == 1];
    tr_y = y_indexes[myarray == 1];
    sq_x = x_indexes[myarray == 2];
    sq_y = y_indexes[myarray == 2];

    # plot the squares and triangles. make the size of the polygons
    # larger than the default so they're easy to see!
    # Note: we have to reverse the y values to make sure it plots with
    plt.plot(tr_x, myarray.shape[0] - tr_y - 1, '^g', markersize=20)
    plt.plot(sq_x, myarray.shape[0] - sq_y - 1, 'rs', markersize=20)
```

```
# Set the x and y limits to include half a space overlap so we don't  
plt.ylim([-0.5, myarray.shape[0] + 0.5])  
plt.xlim([-0.5, myarray.shape[1] + 0.5])  
  
# Turn the axes off  
plt.axis('off')
```

Let's set up the forest and visualize it

```
In [9]: forest = create_forest(6,0.5)  
fig = plt.figure(figsize=(3,3))  
plotgrid(forest)
```



But let's explore some bits and pieces of this

```
In [10]: def create_forest(N=50,f_trees_start=0.5):  
    '''  
    Creates the initial game board.  
    Inputs:  
        board_size: length of one edge of the board  
        f_trees_start: probability that a given cell is a tree  
                       (effectively the tree density)  
  
    Outputs a 2D numpy array with values set to either 0, 1, or 2  
        (empty, tree, or fire)  
    '''  
    # all cells initialized to 'empty' (0) by default; board is defined  
    game_board = np.zeros((N,N))  
    print("the empty board:\n",game_board)  
    print("\n N_rows:",game_board.shape[0], "N_columns:", game_board.shape[1])  
  
    # loop over all rows and columns on the board and roll the dice;  
    # if the random number is less than f_trees_start, make it a tree.  
    for i in range(game_board.shape[0]): # .shape[0] gives us number of rows  
        for j in range(game_board.shape[1]): # .shape[1] gives us the number of columns  
            if random.random() <= f_trees_start:  
                game_board[i,j] = 1  
  
    # set the whole left edge of the board on fire.  
    # This means all rows for the first column, should be set to 2  
    game_board[:,0] = 2  
    print("\n the fully set up board:\n",game_board)
```

```
return game_board
```

In [15]: *# function plotgrid() takes in a 2D array and uses matplotlib to make a
this function returns no values!*

```
def plotgrid(myarray):
```

```
# first create two vectors based on the x and y sizes of the grid  
x_range = np.linspace(0, myarray.shape[1]-1, myarray.shape[1]) # x range  
y_range = np.linspace(0, myarray.shape[0]-1, myarray.shape[0]) # y range
```

```
# use the numpy meshgrid function to create two matrices  
# of the same size as myarray with x and y indexes
```

```
x_indexes, y_indexes = np.meshgrid(x_range, y_range)
```

```
print("\n x from meshgrid:\n",x_indexes)
```

```
print("\n y from meshgrid:\n",y_indexes)
```

```
# make a list of all the x and y indexes that are either squares or triangles  
# the notation below is relatively new to us; it means that when myarray == 1  
# only record those values.
```

```
tr_x = x_indexes[myarray == 1];
```

```
tr_y = y_indexes[myarray == 1];
```

```
sq_x = x_indexes[myarray == 2];
```

```
sq_y = y_indexes[myarray == 2];
```

```
# plot the squares and triangles. make the size of the polygons  
# larger than the default so they're easy to see!
```

```
# Note: we have to reverse the y values to make sure it plots with the correct  
# orientation  
plt.plot(tr_x, myarray.shape[0] - tr_y - 1, '^g',markersize=20)
```

```
plt.plot(sq_x, myarray.shape[0] - sq_y - 1, 'rs', markersize=20)

# What if we didn't do that "reversing" part with the .shape and suc
#plt.plot(tr_x, tr_y, '^g', markersize=20)
#plt.plot(sq_x, sq_y, 'rs', markersize=20)

# Set the x and y limits to include half a space overlap so we don't
plt.ylim([-0.5, myarray.shape[0] + 0.5])
plt.xlim([-0.5, myarray.shape[1] + 0.5])

# Turn the axes off
plt.axis('off')
```

```
In [16]: forest = create_forest(6, 0.5)
fig = plt.figure(figsize=(3, 3))
plotgrid(forest)
```

the empty board:

```
[[0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0.]]
```

N_rows: 6 N_columns:: 6

the fully set up board:

```
[[2. 0. 1. 1. 0. 1.]  
 [2. 0. 1. 0. 1. 0.]  
 [2. 1. 1. 1. 0. 0.]  
 [2. 1. 0. 0. 0. 0.]  
 [2. 1. 0. 0. 1. 0.]  
 [2. 1. 0. 0. 1. 0.]]
```

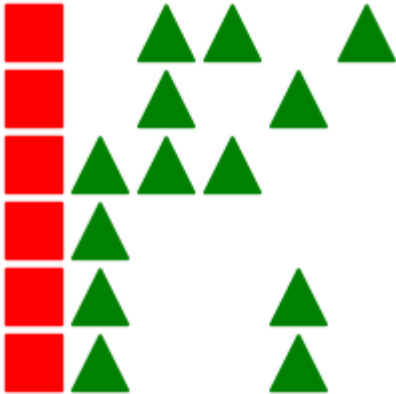
x from meshgrid:

```
[[0. 1. 2. 3. 4. 5.]  
 [0. 1. 2. 3. 4. 5.]  
 [0. 1. 2. 3. 4. 5.]  
 [0. 1. 2. 3. 4. 5.]  
 [0. 1. 2. 3. 4. 5.]  
 [0. 1. 2. 3. 4. 5.]]
```

y from meshgrid:

```
[[0. 0. 0. 0. 0. 0.]  
 [1. 1. 1. 1. 1. 1.]  
 [2. 2. 2. 2. 2. 2.]  
 [3. 3. 3. 3. 3. 3.]]
```

```
[4. 4. 4. 4. 4. 4.]  
[5. 5. 5. 5. 5. 5.]
```



But what if we didn't have a square board?

```
In [17]: def create_forest(M=30, N=50, f_trees_start=0.5):  
# all cells initialized to 'empty' (0) by default  
# board is defined to be M x N  
game_board = np.zeros((M,N))  
print("the empty board:\n",game_board)  
print("\n M_rows:",game_board.shape[0], "N_columns:", game_board.sl  
  
# loop over all rows and columns on the board and roll the dice;  
# if the random number is less than f_trees_start, make it a tree.  
for i in range(game_board.shape[0]): # .shape[0] gives us number of
```

```
        for j in range(game_board.shape[1]): # .shape[1] gives us the number of columns
            if random.random() <= f_trees_start:
                game_board[i,j] = 1

# set the whole left edge of the board on fire.
# This means all rows for the first column, should be set to 2
game_board[:,0] = 2
print("\n the fully set up board:\n",game_board)

return game_board
```

```
In [18]: forest = create_forest(3,6,0.5)
fig = plt.figure(figsize=(3,1.5))
plotgrid(forest)
```

```
the empty board:
[[0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]]
```

```
M_rows: 3 N_columns:: 6
```

the fully set up board:

```
[[2. 1. 1. 0. 0. 0.]
 [2. 0. 1. 0. 1. 1.]
 [2. 1. 1. 0. 0. 1.]]
```

- Continue manipulating 2D NumPy arrays to store data and build models

• Lay the groundwork for the forest fire model.

```
x from meshgrid:
[[0. 1. 2. 3. 4. 5.]
 [0. 1. 2. 3. 4. 5.]
 [0. 1. 2. 3. 4. 5.]]
```

- Need to make sure we have functional tools for setting up and visualizing the model
- Need to create a function that can advance the model **one timestep**

y from meshgrid:
forward in time.

```
[[0. 0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1. 1.]
 [2. 2. 2. 2. 2. 2.]]
```

Your functions `onBoard` and `getNeighborValues` from last class will be useful!

- The work you do today will set us up to run and explore the properties of model on

Wednesday.



Looking forward - Day 19 PCA: Continuing work with NumPy arrays, but in a new context.

- Learn a bit about voting theory
- Use NumPy as a way of storing votes and determining election outcomes

